

Subfinder Tool - Tester Workflow Report

Step-by-Step Execution Process for Reconnaissance Testing

Purpose

This report documents the standard step-by-step process a security tester follows when using **Subfinder** for reconnaissance (subdomain discovery), from initial setup through to final reporting. Each step includes the situation in which it applies and the corresponding command, followed by a decision guide for choosing between options mid-engagement.

What Is Subfinder?

Subfinder is a fast, open-source passive subdomain discovery tool developed by ProjectDiscovery. Unlike Amass (which is a broader attack-surface mapping suite), Subfinder is purpose-built to do one thing very well: enumerate subdomains quickly using passive sources, with minimal noise and high accuracy. It's commonly used as an early, lightweight recon step — often before or alongside a heavier tool like Amass.

Step 1: Scoping and Authorization

Situation: Before any scanning begins, the tester confirms the target domain is owned by the client, covered by a signed scope-of-work/authorization document, or is a publicly designated practice target.

Action: No command yet — this is a documentation/legal step: confirm the target domain(s), confirm passive-only vs. active-allowed scope, and get written sign-off.

Step 2: Basic Passive Enumeration

Situation: This is the **first scan run on any target**. Subfinder is passive by default — it never sends traffic directly to the target, only querying public sources (certificate transparency logs, DNS aggregators, search engines, APIs).

Command:

```
subfinder -d example.com
```

What it does: Queries all configured passive sources and prints discovered subdomains directly to the terminal. This is the simplest possible run and the standard starting point for any engagement.

Step 3: Saving Results to a File

Situation: Once the tester confirms the scan is producing useful output, results need to be saved for later analysis and reporting, rather than just printed to the screen.

Command:

```
subfinder -d example.com -o results.txt
```

When to use it: Every real engagement — raw terminal output isn't sufficient for a client report or for feeding into the next tool in the pipeline.

Step 4: Increasing Output Detail

Situation: The tester wants more context than a plain subdomain list — for example, to see which source found each result, for credibility and troubleshooting.

Command:

```
subfinder -d example.com -v -o results.txt
```

Situation-specific note: `-v` (verbose) prints extra diagnostic information, including the exact behavior of each source, which is useful when a client asks "how was this subdomain found?"

Step 5: Structured Output for Automation

Situation: Results need to be fed into another tool (like a port scanner or HTTP prober) as part of an automated pipeline, rather than read by a human.

Command:

```
subfinder -d example.com -oJ -o results.json
```

When to use it: Any time Subfinder is one step in a larger automated recon chain (a very common setup, e.g., `subfinder -> httpx -> nuclei`).

Step 6: Multiple Domains at Once

Situation: The client's scope includes several domains or brands, and the tester wants to avoid running the command manually for each one.

Command:

```
subfinder -dL domains.txt -o results.txt
```

When to use it: Any multi-domain engagement — `domains.txt` contains one target domain per line.

Step 7: Configuring API Keys for Better Coverage

Situation: Some of Subfinder's best data sources (SecurityTrails, Censys, Shodan, VirusTotal, etc.) require API keys for full results; without them, those sources are skipped or heavily rate-limited.

Command:

```
subfinder -d example.com -config config.yaml -o results.txt
```

Situation-specific note: Testers on a real engagement typically configure this once at the start of a project rather than per-scan, since API keys are usually reused across the whole assessment.

Step 8: Filtering to Only Active/Resolvable Subdomains

Situation: A raw subdomain list often includes stale or dead entries (subdomains that used to exist but no longer resolve). Before deeper testing, the tester wants only the ones that are currently live.

Command:

```
subfinder -d example.com -o results.txt -nW
```

Alternative — chaining with a DNS resolver:

```
subfinder -d example.com -silent | dnsx -silent -o live_subdomains.txt
```

When to use it: Right before handing results off to active scanning tools, to avoid wasting time probing dead hosts.

Step 9: Documenting and Reporting Results

Situation: The final step in every engagement, regardless of how many subdomains were found. The tester records:

- Exact command(s) and flags used
 - Number of subdomains discovered
 - Which sources contributed results (if `-v` was used)
 - Any errors, rate-limiting issues, or missing API key warnings
 - Output file(s) produced, ready to hand off to the next phase (e.g., Amass active enumeration, port scanning, or HTTP probing)
-

Decision Guide — Which Command Does the Tester Choose, and Why?

At several points in an engagement, the tester has to make a judgment call about which Subfinder command or option to run next. This section covers those decision points.

Situation the Tester Encounters	Decision & Reasoning	Command Used
First time running the tool on this target, just want a quick look	Run the simplest possible passive scan, no flags needed	<code>subfinder -d example.com</code>
Results need to be kept for a report or handed to another tool	Always save output to a file rather than relying on terminal scrollback	<code>subfinder -d example.com -o results.txt</code>
Client or manager asks how a specific subdomain was discovered	Re-run with verbose mode so each result is tagged with its source	<code>subfinder -d example.com -v -o results.txt</code>
Output will be consumed by another automated tool, not read by a person	Switch to structured JSON output instead of plain text	<code>subfinder -d example.com -oJ -o results.json</code>
Client's scope includes multiple domains or brands	Use a domain list file instead of running the command once per domain	<code>subfinder -dL domains.txt -o results.txt</code>
Initial passive scan returns fewer results than expected for a large organization	Configure API keys for premium sources (SecurityTrails, Censys, etc.) and re-run	<code>subfinder -d example.com -config config.yaml -o results.txt</code>
Tester is about to hand results to an active scanner and wants to avoid wasting time on dead hosts	Filter the list down to only currently-resolvable subdomains before moving on	<code>subfinder -d example.com -silent dnsx -silent -o live_subdomains.txt</code>
Engagement scope explicitly limits testing to passive-only techniques	Stick to default Subfinder behavior — it's passive by design, so no extra flags are needed to stay in scope	<code>subfinder -d example.com -o results.txt</code>
Subfinder needs to feed directly into a second recon tool (e.g., Amass) for a deeper pass	Output in silent mode (clean list, no banners) so it pipes cleanly into the next command	<code>subfinder -d example.com -silent -o subs.txt</code>
Tester wants to double-check coverage against a heavier tool later in the engagement	Keep Subfinder's output separate and compare it against Amass results afterward, rather than treating either as final on its own	<i>(no single command — a comparison step, not a scan)</i>

General decision principle the tester follows: Subfinder is almost always run passively by default, so the main decisions are about *output format* (human-readable vs. automation-ready) and *coverage* (default sources vs. API-key-enabled sources) rather than about escalating intrusiveness, since Subfinder doesn't have an active/brute-force mode the way Amass does.

Summary Table — Full Tester Journey

#	Step	Situation	Example Command
1	Scoping & authorization	Before any scan	<i>(no command — approval step)</i>
2	Basic passive enumeration	First scan on any target	<code>subfinder -d example.com</code>
3	Save results to file	Need output for reporting	<code>subfinder -d example.com -o results.txt</code>
4	Verbose/source-tagged output	Need to show source of each finding	<code>subfinder -d example.com -v -o results.txt</code>
5	Structured JSON output	Feeding into another tool	<code>subfinder -d example.com -oJ -o results.json</code>
6	Multiple domains	Several brands/domains in scope	<code>subfinder -dL domains.txt -o results.txt</code>
7	API key configuration	Need deeper coverage	<code>subfinder -d example.com -config config.yaml -o results.txt</code>
8	Filter to live subdomains	Preparing for active scanning	<code>subfinder -d example.com -silent \ dnsx -silent -o live_subdomains.txt</code>
9	Reporting	Always, at the end	This report

This report describes standard Subfinder usage for a passive subdomain-discovery phase, typically run before or alongside deeper tools like Amass, and only ever against domains that are explicitly in scope and authorized for testing.